Name: Lance Timothy Satorre
Github Link: https://github.com/Lancesatorre/IntProgActivities/tree/main/Lab6-Activity_SATORRE/node-mysql-api

Node.js + TypeScript + MySQL
Authentication API

Quick Reference & Recreation Guide

Express · Sequelize · MySQL · JWT · Nodemailer · Swagger

What We're Building

A production-ready authentication REST API with user registration, JWT login, role-based access, and account management.

- User registration with email verification
- JWT login (15 min) + HTTP-Only refresh token cookies (7 days)
- Refresh token rotation for extra security
- Admin and User roles (first account = Admin)
- Forgot / reset password via email
- Swagger docs at /api-docs

Project Structure

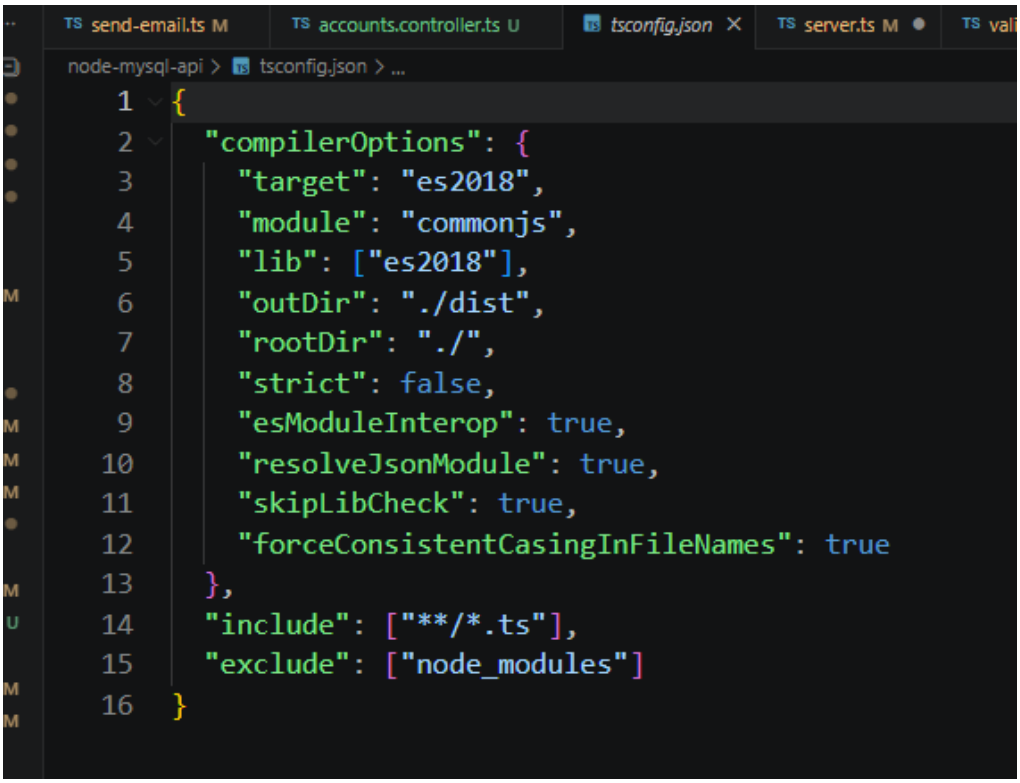
```
project-root/
├── _helpers/
│   ├── db.ts          ← MySQL + Sequelize connection
│   ├── send-email.ts  ← Nodemailer wrapper
│   ├── role.ts        ← Role enum (Admin / User)
│   └── swagger.ts     ← Swagger UI route
├── _middleware/
│   ├── authorize.ts        ← JWT + role guard
│   ├── error-handler.ts    ← Global error handler
│   └── validate-request.ts ← Joi body validation
├── accounts/
│   ├── account.model.ts        ← Sequelize Account model
│   ├── refresh-token.model.ts  ← Sequelize RefreshToken model
│   ├── account.service.ts      ← Business logic
│   └── accounts.controller.ts  ← Express routes
├── config.json   ← DB + JWT + SMTP settings
├── swagger.yaml  ← API spec
└── server.ts     ← App entry point
```

Part 1 — Project Setup

1.1 Initialise & Install

```
npm init -y

# Production
npm install express sequelize mysql2 jsonwebtoken bcryptjs nodemailer
            express-jwt joi swagger-ui-express yamljs cookie-parser cors
rootpath

# Dev
npm install -D typescript ts-node nodemon @types/node @types/express
            @types/jsonwebtoken @types/bcryptjs @types/nodemailer
```

1.2 tsconfig.json



1.3 package.json scripts

node-mysql-api > {} package.json > ...

```
{
  "name": "mysql-api",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  ▷Debug
  "scripts": {
    "start": "ts-node ./server.ts",
    "start:dev": "nodemon --exec ts-node ./server.ts"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "body-parser": "^2.2.2",
    "cookie-parser": "^1.4.7",
    "cors": "^2.8.6",
    "express": "^5.2.1",
    "express-jwt": "^8.5.1",
    "joi": "^18.1.2",
    "jsonwebtoken": "^9.0.3",
    "mysql2": "^3.22.0",
    "nodemailer": "^8.0.5",
    "sequelize": "^6.37.8",
    "swagger-ui-express": "^5.0.1",
    "yamljs": "^0.3.0"
  },
  "devDependencies": {
    "@types/bcryptjs": "^2.4.6",
    "@types/cookie-parser": "^1.4.10",
    "@types/cors": "^2.8.19",
    "@types/express": "^5.0.6",
    "@types/jsonwebtoken": "^9.0.10",
    "@types/node": "^25.6.0",
    "@types/nodemailer": "^8.0.0",
    "@types/swagger-ui-express": "^4.1.8",
    "@types/yamljs": "^0.2.34",
    "nodemon": "^3.1.14",
    "ts-node": "^10.9.2",
    "typescript": "^6.0.2"
  }
}
```

Part 2 — config.json

Create this file in the project root. Get free SMTP credentials at ethereal.email.

TS send-email.ts M | TS accounts.controller.ts U | {} package.json | TS server.ts M | TS va

node-mysql-api > {} config.json > {} smtpOptions > {} auth > abc pass

```
{
    "database": {
        "host": "localhost",
        "port": 3306,
        "user": "root",
        "password": "lance123",
        "database": "node_mysql_api"
},
    "secret": "SUPER_SECRET_KEY_REPLACE_ME_IN_PRODUCTION",
    "emailFrom": "info@my-node-api.com",
    "smtpOptions": {
        "host": "smtp.ethereal.email",
        "port": 587,
        "auth": {
            "user": "hellen40@ethereal.email",
            "pass": "f66wDGACN9uMMYYARC"
        }
    }
}
```

Part 3 — Helper Files

_helpers/db.ts

Connects to MySQL, initialises Sequelize, loads models, syncs tables on startup.

```
TS send-email.ts M   TS accounts.controller.ts U   TS server.ts M   TS validate-request.ts M   {} config.json M
node-mysql-api > src > _helpers > TS db.ts > initialize
1   import config from '../../config.json';
2   import mysql from 'mysql2/promise';
3   import { Sequelize } from 'sequelize';
4   import accountModel from '../accounts/account-model';
5   import refreshTokenModel from '../accounts/refresh-token.model';
6
7   const db: any = {};
8   export default db;
9
10  initialize();
11
12  async function initialize() {
13      const appConfig: any = config;
14      const { host, port, user, password, database } = appConfig.database;
15      const connection = await mysql.createConnection({ host, port, user, password });
16
17      // Create DB if it doesn't exist
18      await connection.query(`CREATE DATABASE IF NOT EXISTS \`${database}\`;`);
19
20      // Connect to DB
21      const sequelize = new Sequelize(database, user, password, { dialect: 'mysql' });
22
23      // Init models
24      db.Account = accountModel(sequelize);
25      db.RefreshToken = refreshTokenModel(sequelize);
26
27      // Define relationships
28      db.Account.hasMany(db.RefreshToken, { onDelete: 'CASCADE' });
29      db.RefreshToken.belongsTo(db.Account);
30
31      // Sync models with database
32      await sequelize.sync();
33  }
```

_helpers/send-email.ts

Thin Nodemailer wrapper — reads SMTP from config.json, exports sendEmail().

```
node-mysql-api > src > _helpers > TS send-email.ts > sendEmail
1   import nodemailer from 'nodemailer';
2   import config from '../../config.json';
3
4   export default async function sendEmail({ to, subject, html, from = config.smtpOptions.auth.user || config.emailFrom }: any) {
5     try {
6       console.log('Creating SMTP transporter with host:', config.smtpOptions.host);
7       const transporter = nodemailer.createTransport(config.smtpOptions);
8       await transporter.verify();
9
10      console.log('Sending email to:', to);
11      const result = await transporter.sendMail({ from, to, subject, html });
12      console.log('Email sent successfully:', result);
13      const previewUrl = nodemailer.getTestMessageUrl(result);
14      if (previewUrl) {
15        console.log('Ethereal preview URL:', previewUrl);
16      }
17      return result;
18    } catch (error: any) {
19      console.error('Email sending failed:', error.message);
20      throw error;
21    }
22  }
```

_helpers/role.ts

```
export const Role = { Admin: 'Admin', User: 'User' };
```

node-mysql-api > src > _helpers > role.ts > default

```
export default {
    Admin: 'Admin',
    User: 'User'
}
```

_helpers/swagger.ts

Serves auto-generated Swagger UI from swagger.yaml at the /swagger route.

node-mysql-api > src > _helpers > swagger.ts > default

```
import express from 'express';
const router = express.Router();
import swaggerUi from 'swagger-ui-express';
import YAML from 'yamljs';

const swaggerDocument = YAML.load('./swagger.yaml');

router.use('/', swaggerUi.serve, swaggerUi.setup(swaggerDocument));

export default router;
```

Part 4 — Middleware

_middleware/error-handler.ts

Global error catcher. String errors → 400 (or 404 if ending in 'not found'). No try-catch needed in controllers.

node-mysql-api > src > _middlerware > error-handler.ts > ...

```
import { Request, Response, NextFunction } from 'express';

export default function errorHandler(err: any, req: Request, res: Response, next: NextFunction) {
    console.error('Error:', err);

    switch (true) {
        case typeof err === 'string':
            // custom application error
            const is404 = err.toLowerCase().endsWith('not found');
            const statusCode = is404 ? 404 : 400;
            console.log('Sending error response:', statusCode, err);
            return res.status(statusCode).json({ message: err });
        case err.name === 'UnauthorizedError':
            // jwt authentication error
            return res.status(401).json({ message: 'Unauthorized' });
        default:
            console.log('Sending 500 error response:', err.message);
            return res.status(500).json({ message: err.message });
    }
}
```

_middleware/validate-request.ts

Validates req.body against a Joi schema. Returns 400 with details if invalid.

node-mysql-api > src > _middlerware > validate-request.ts > validateRequest

```
export default validateRequest;

function validateRequest(req: any, next: any, schema: any) {
    const options = {
        abortEarly: false,
        allowUnknown: true,
        stripUnknwon: true
    };

    console.log('Validating request body:', req.body);
    const {error, value} = schema.validate(req.body, options);

    if (error) {
        console.log('Validation error:', error.details.map(x => x.message).join(', '
        next(`Validation error: ${error.details.map(x => x.message).join(', ')}`);
    } else {
        console.log('Validation passed');
        req.body = value;
        next();
    }
}
```

_middleware/authorize.ts

Protects routes. Verifies JWT Bearer token, checks account still exists, enforces role. Returns 401 on failure.

node-mysql-api > src > _middlerware > authorize.ts > ...

```
import { expressjwt as jwt } from 'express-jwt';
import config from '../../config.json';
import db from '../_helpers/db';


const appConfig: any = config;
const { secret } = appConfig;

export default function authorize(roles: any = []) {
    if (typeof roles === 'string') {
        roles = [roles];
    }

    return [
        jwt({ secret, algorithms: ['HS256'] }),
        async (req: any, res: any, next: any) => {
            // express-jwt v8 stores JWT payload on req.auth by default.
            const tokenUser = req.auth || req.user;
            const accountId = tokenUser?.sub || tokenUser?.id;
            const account = await db.Account.findByPk(accountId);

            if (!account || (roles.length && !roles.includes(account.role))) {
                return res.status(401).json({ message: 'Unauthorized' });
            }

            req.user = req.user || {};
            req.user.id = account.id;
            req.user.role = account.role;
            const refreshTokens = await account.getRefreshTokens();
            req.user.ownsToken = (token: any) => !!refreshTokens.find((x: any) => x.token === token);
            next();
        }
    ];
}
```

Part 5 — Accounts Module

account.model.ts

Sequelize schema for the accounts table. Virtual fields (isVerified) are computed, not stored. defaultScope hides passwordHash by default.

node-mysql-api > src > accounts > account-model.ts > model > options > scopes

```typescript
import { DataTypes } from 'sequelize';

export default function model(sequelize: any) {
    const attributes = {
        email: { type: DataTypes.STRING, allowNull: false },
        passwordHash: { type: DataTypes.STRING, allowNull: false },
        title: { type: DataTypes.STRING, allowNull: false },
        firstName: { type: DataTypes.STRING, allowNull: false },
        lastName: { type: DataTypes.STRING, allowNull: false },
        acceptTerms: { type: DataTypes.BOOLEAN },
        role: { type: DataTypes.STRING, allowNull: false },
        verificationToken: { type: DataTypes.STRING },
        verified: { type: DataTypes.DATE },
        resetToken: { type: DataTypes.STRING },
        resetTokenExpires: { type: DataTypes.DATE },
        passwordReset: { type: DataTypes.DATE },
        created: { type: DataTypes.DATE, allowNull: false, defaultValue: DataTypes.NOW },
        updated: { type: DataTypes.DATE },
        isVerified: {
            type: DataTypes.VIRTUAL,
            get() { return !!(this.verified || this.passwordReset); }
        }
    };

    const options = {
        timestamps: false,
        defaultScope: {
            // exclude password hash by default
            attributes: { exclude: ['passwordHash'] }
        },
        scopes: {
            // include hash with this scope
            withHash: { attributes: {}, }
        }
    };

    return sequelize.define('account', attributes, options);
}
```

refresh-token.model.ts

Sequelize schema for refreshTokens. Virtual fields isExpired and isActive computed at runtime.

node-mysql-api > src > accounts > refresh-token.model.ts > ...

```typescript
import { DataTypes } from 'sequelize';

export default function model(sequelize: any) {
    const attributes = {
        token: { type: DataTypes.STRING },
        expires: { type: DataTypes.DATE },
        created: { type: DataTypes.DATE, allowNull: false, defaultValue: DataTypes.NOW },
        createdByIp: { type: DataTypes.STRING },
        revoked: { type: DataTypes.DATE },
        revokedByIp: { type: DataTypes.STRING },
        replacedByToken: { type: DataTypes.STRING },
        isExpired: {
            type: DataTypes.VIRTUAL,
            get() { return Date.now() >= this.expires; }
        },
        isActive: {
            type: DataTypes.VIRTUAL,
            get() { return !this.revoked && !this.isExpired; }
        }
    };

    const options = { timestamps: false };
    return sequelize.define('refreshToken', attributes, options);
}
```

account.service.ts

All business logic. The only layer that touches the database. Key methods:

- authenticate() — verify credentials, return JWT + refresh token
- refreshToken() — rotate token, return new JWT
- register() — hash password, create account, send verification email
- verifyEmail() — mark account as verified
- forgotPassword() / resetPassword() — email-based password reset
- getAll() / getById() / create() / update() / delete() — CRUD

node-mysql-api > src > accounts > TS account.service.ts > register

```
import config from '../../config.json';
import jwt from 'jsonwebtoken';
import bcrypt from 'bcryptjs';
import crypto from 'crypto';
import sendEmail from '../_helpers/send-email';
import { Op } from 'sequelize';
import db from '../_helpers/db';
import Role from '../_helpers/role';
```

```
async function authenticate({email, password, ipAddress}: any) {
    const account = await db.Account.scope('withHash').findOne({where: {email}});

    if (!account || !account.isVerified || ! (await bcrypt.compare(password, account.passwordHash))) {
        throw 'Email or password is incorrect';
    }

    const jwtToken = generateJwtToken(account);
    const refreshToken = generateRefreshToken(account, ipAddress);

    await refreshToken.save();

    return {
        ...basicDetails(account),
        jwtToken,
        refreshToken: refreshToken.token
    };
}
```

```
async function register(params: any, origin: any){
    if (await db.Account.findOne({where: {email: params.email}})) {
        try {
            await sendAlreadyRegisteredEmail(params.email, origin);
        } catch (error) {
            console.error('Error sending already registered email:', error);
        }
        throw 'Email already registered';
    }

    const account = new db.Account(params);

    const isFirstAccount = (await db.Account.count()) === 0;
    account.role = isFirstAccount ? Role.Admin : Role.User;
    account.verificationToken = randomTokenString();

    account.passwordHash = await hash(params.password);

    await account.save();

    // Do not swallow email delivery failures; registration should fail if verification email cannot be sent.
    await sendVerificationEmail(account, origin);
}
```

accounts.controller.ts

All /accounts routes. Route definitions at the top, handlers below. Uses authorize() for protected routes and Joi schemas for body validation.

node-mysql-api > src > accounts > TS accounts.controller.ts > refreshToken

```typescript
import express from 'express';
const router = express.Router();
import Joi, { func } from 'joi';
import validateRequest from '../_middlerware/validate-request';
import authorize from '../_middlerware/authorize';
import Role from '../_helpers/role';
import accountService from './account.service';

router.post('/authenticate', authenticateSchema, authenticate);
router.post('/refresh-token', refreshToken);
router.post('/revoke-token', authorize(), revokeTokenSchema, revokeToken);
router.post('/register', registerSchema, register);
router.post('/verify-email', verifyEmailSchema, verifyEmail);
router.post('/forgot-password', forgotPasswordSchema, forgotPassword);
router.post('/validate-reset-token', validateResetTokenSchema, validateResetToken);
router.post('/reset-password', resetPasswordSchema, resetPassword);
router.get('/', authorize(Role.Admin), getAll);
router.get('/:id', authorize(), getById);
router.post('/', authorize(Role.Admin), createSchema, create);
router.put('/:id', authorize(), updateSchema, update);
router.delete('/:id', authorize(), _delete);

export default router;
```

```typescript
function authenticateSchema(req: any, res: any, next: any) {
    const schema = Joi.object({
        email: Joi.string().required(),
        password: Joi.string().required()
    });

    validateRequest(req,next,schema);
}

function authenticate(req: any, res: any, next: any) {
    const {email, password} = req.body;
    const ipAddress = req.ip;

    accountService.authenticate({email, password, ipAddress})
        .then(({refreshToken, ...account}: any) => {
            setTokenCookie(res, refreshToken);
            res.json(account);
        })
        .catch(next);
}
```

```
function registerSchema(req: any, res: any, next: any) {
    const schema = Joi.object({
        title: Joi.string().required(),
        firstName: Joi.string().required(),
        lastName: Joi.string().required(),
        email: Joi.string().email().required(),
        password: Joi.string().min(6).required(),
        confirmPassword: Joi.string().valid(Joi.ref('password')).required(),
        acceptTerms: Joi.boolean().valid(true).required()
    });
    validateRequest(req, next, schema);
}

function register(req: any, res: any, next: any) {
    console.log('Register endpoint called with email:', req.body.email);
    accountService.register(req.body, req.get('origin'))
        .then(() => {
            console.log('Registration successful for email:', req.body.email);
            res.json({message: 'Registration successful, please check your email for verification instructions'});
        })
        .catch(next);
}
```

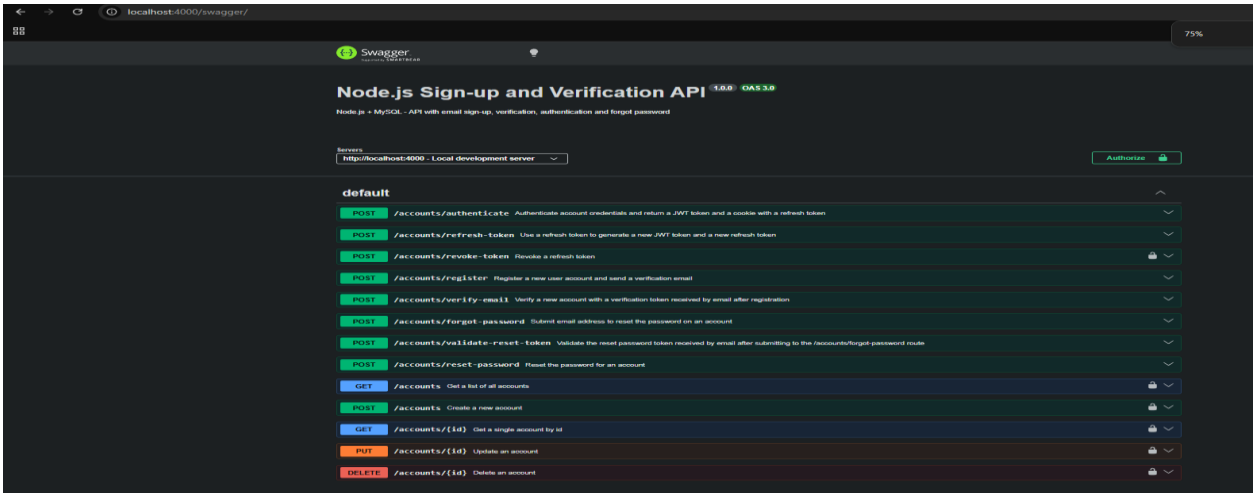
Part 6 — server.ts

```
node-mysql-api > TS server.ts > ...
import express from 'express';
import bodyParser from 'body-parser';
import cookieParser from 'cookie-parser';
import cors from 'cors';
import errorHandler from './src/_middlerware/error-handler';
import accountsController from './src/accounts/accounts.controller';
import swaggerDocs from './src/_helpers/swagger';

const app = express();

app.use(bodyParser.urlencoded({ extended: false }));
app.use(bodyParser.json());
app.use(cookieParser());

// allow cors requests from any origin and with credentials
app.use(cors({ origin: (origin, callback) => callback(null, true), credentials: true }));

// api routes
app.use('/accounts', accountsController);

// swagger docs route
app.use('/swagger', swaggerDocs);

// global error handler
app.use(errorHandler);

// start server
const port = process.env.NODE_ENV === 'production' ? (process.env.PORT || 80) : 4000;
app.listen(port, () => console.log('Server listening on http://localhost:4000/ '));
```

Start the dev server:

```
npm run dev
```

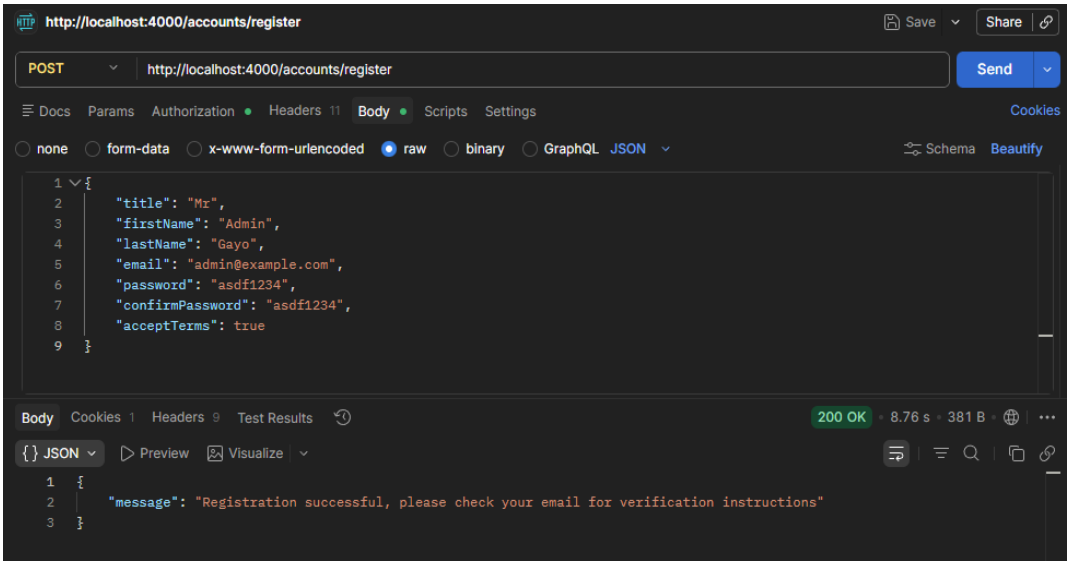
```
> mysql-api@1.0.0 start
> ts-node ./server.ts

Server listening on http://localhost:4000/
Executing (default): SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE TABLE' AND TABLE_NAME = 'accounts' AND TABLE_SCHEMA = 'node_mysql_api'
Executing (default): SHOW INDEX FROM `accounts`
Executing (default): SELECT TABLE_NAME FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_TYPE = 'BASE TABLE' AND TABLE_NAME = 'refreshTokens' AND TABLE_SCHEMA = 'node_mysql_api'
Executing (default): SHOW INDEX FROM `refreshTokens`
```



Part 7 — Testing with Postman

7.1 Register

1. New tab → POST → http://localhost:4000/accounts/register
2. Body → raw → JSON → paste the object below
3. Send → expect 200 OK 'registration successful'



7.2 Verify Email

Check Ethereal inbox → copy the token from the email.

To: <admin@example.com>	Sign-up Verification API - Verify Email	Today at 15:52
★ Ethereal Email <info@ethereal.email>	This is your ethereal mailbox	Today at 15:52

Public URL of this message

Subject: Sign-up Verification API - Verify Email
From: <adolphus.cruickshank@ethereal.email>
To: <admin@example.com>
Time: Today at 15:52
Message-ID: <8cf56dfc-2f80-79d0-5cb0-230f4ba55e88@ethereal.email>

HTML Plaintext

Verify Email

Please use the below token to verify your email address with the `/account/verify-email` API route:

`489e7cd4a0e26654e53e183482c3f6f24627b8b68330b21354785ab4719cc640a76546761c915980`

Thanks for registering!

Please use the below token to verify your email address with the `/account/verify-email` API route:

`489e7cd4a0e26654e53e183482c3f6f24627b8b68330b21354785ab4719cc640a76546761c915980`

1. POST → http://localhost:4000/accounts/verify-email
2. Body: { "token": "PASTE_TOKEN" }
3. Send → 200 OK 'verification successful'

POST http://localhost:4000/accounts/verify-email

Docs Params Authorization Headers 11 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
    "token": "489e7cd4a0e26654e53e183482c3f6f24627b8b68330b21354785ab4719cc640a76546761c915980"
}
```

Body Cookies 1 Headers 9 Test Results 200 OK

{} JSON Preview Visualize

```
{
    "message": "Email verified successfully"
}
```

7.3 Login (Authenticate)

1	POST → http://localhost:4000/accounts/authenticate
2	Body: { "email":"john@example.com", "password":"Pass123!" }
3	Send → copy the jwtToken from the response body
4	Check the Cookies tab for the refreshToken cookie

http://localhost:4000/accounts/authenticate

POST http://localhost:4000/accounts/authenticate

Docs Params Authorization Headers 11 Body Scripts Settings

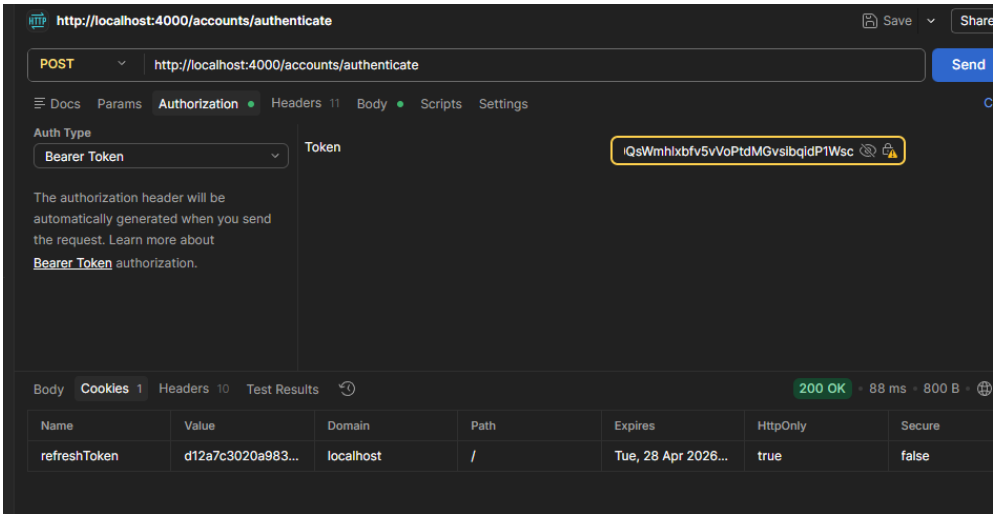
none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
    "email": "admin@example.com",
    "password": "asdf1234"
}
```

Body Cookies 1 Headers 10 Test Results

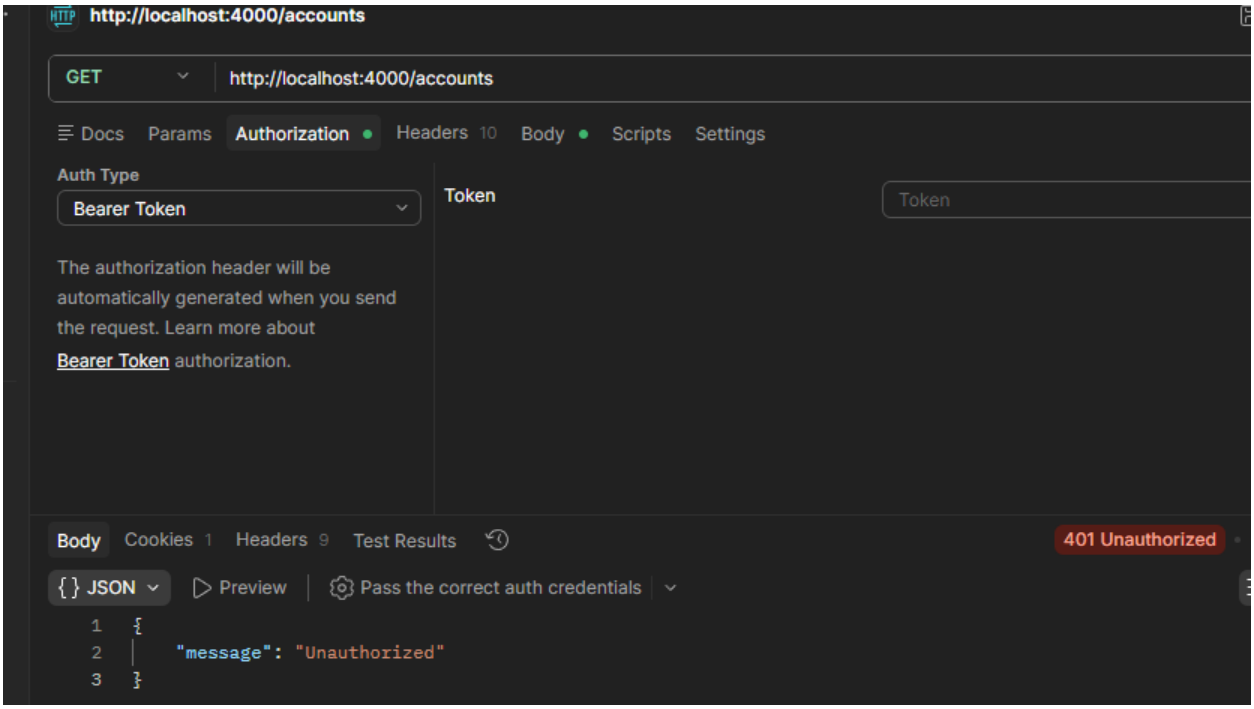
{} JSON Preview Visualize

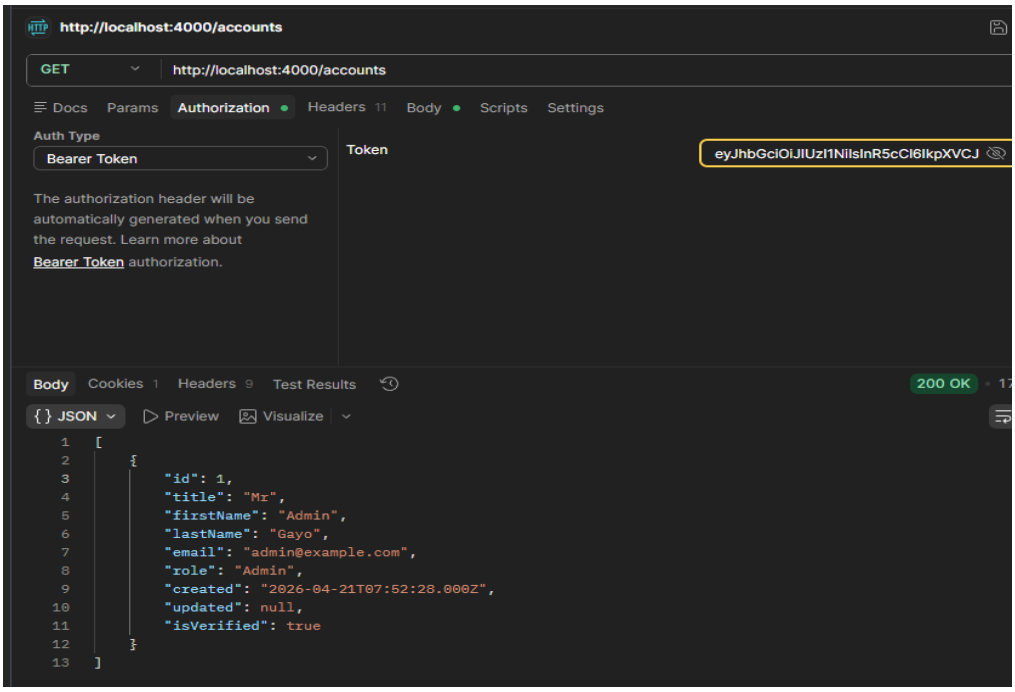

```
{
    "id": 1,
    "title": "Mr",
    "firstName": "Admin",
    "lastName": "Gayo",
    "email": "admin@example.com",
    "role": "Admin",
    "created": "2026-04-21T07:52:28.000Z",
    "updated": null,
    "isVerified": true,
    "jwtToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
        eyJzdWIiOjEsInJvbGUiOiJBZG1pbiIsImlhdCI6MTc3Njc1ODE5NiwiZXhwIjoxNzc2NzU5MDk2fQ.
        nwW3JOQvoCB0QsWmhlxbfv5vVoPtdMGvsibqidP1Wsc"
}
```



7.4 Get All Accounts (Admin only)

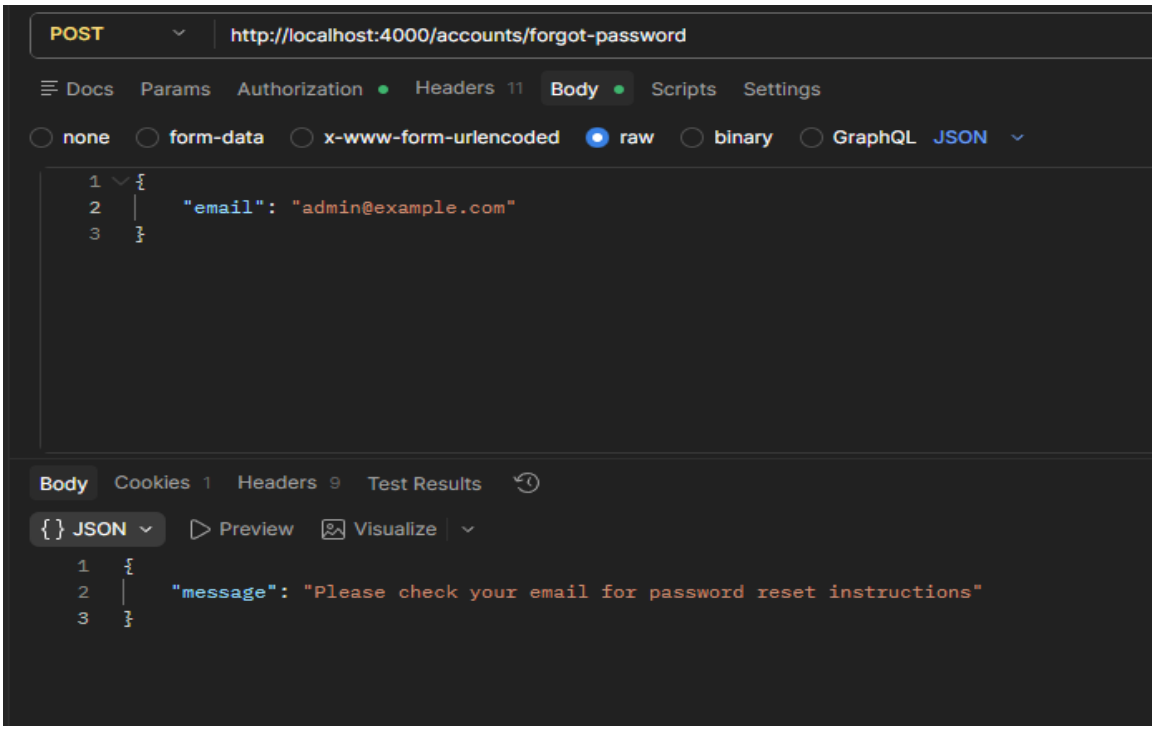
1. GET → http://localhost:4000/accounts
2. Authorization → Bearer Token → paste JWT
3. Send → 200 OK with array of accounts





7.5 Forgot & Reset Password

1	POST → /accounts/forgot-password Body: { "email":"..." }
2	Check Ethereal for the reset email → copy the token
3	POST → /accounts/reset-password Body: { token, password, confirmPassword }



Ethereal

Messages for **adolphus.cruickshank@ethereal.email** – this account can **not** be used for inbound emails

Page **1**

← Newer | Older →

To: <admin@example.com>	**Sign-up Verification API - Reset Password**	**Today at 15:59**
To: <admin@example.com>	Sign-up Verification API - Verify Email	Today at 15:52
★ **Ethereal Email <info@ethereal.email>**	**This is your ethereal mailbox**	**Today at 15:52**

© 2017—2022 Ethereal Email info@ethereal.email | service provided by EmailEngine

Ethereal

Headers | Envelope | Source

Public URL of this message

Subject: Sign-up Verification API - Reset Password
From: <adolphus.cruickshank@ethereal.email>
To: <admin@example.com>
Time: Today at 15:59
Message-ID: <0450e597-1cf7-e67a-22ba-32f9db3a3dd5@ethereal.email>

◉ HTML ○ Plaintext

Reset Password

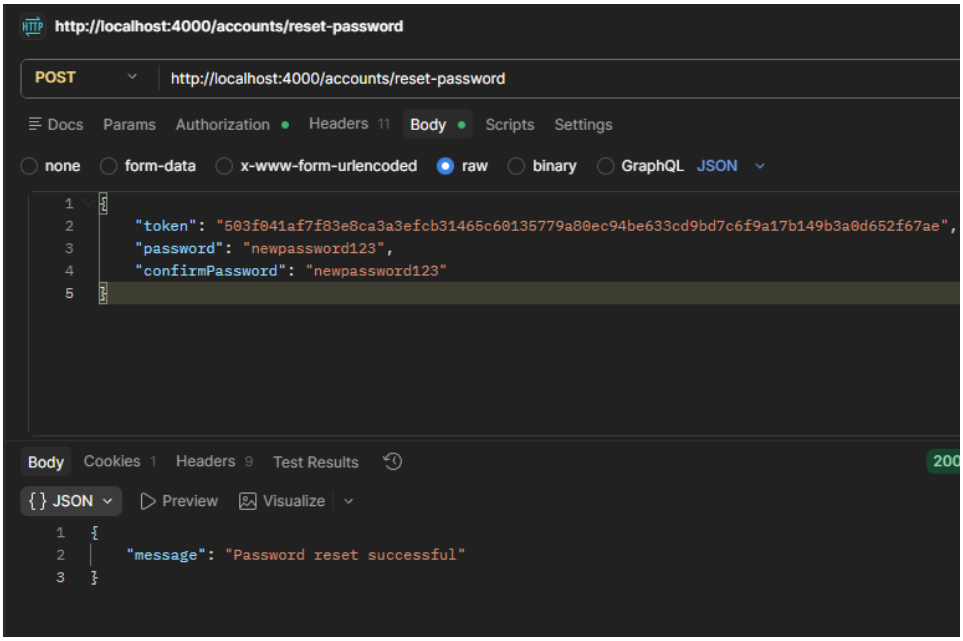
Please use the below token to reset your password with the `/account/reset-password` API route:

`503f041af7f83e8ca3a3efcb31465c60135779a80ec94be633cd9bd7c6f9a17b149b3a0d652f67ae`

If you did not request a password reset, please ignore this email.

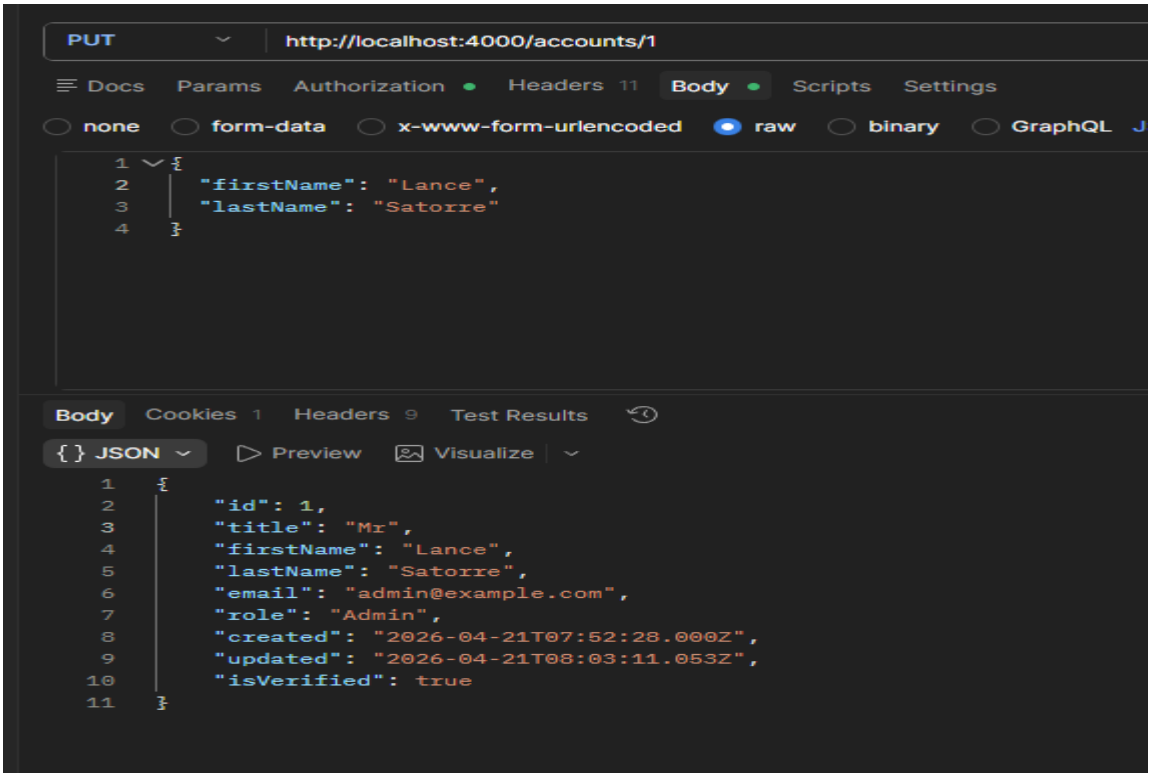
Please use the below token to reset your password with the `/account/reset-password` API route:

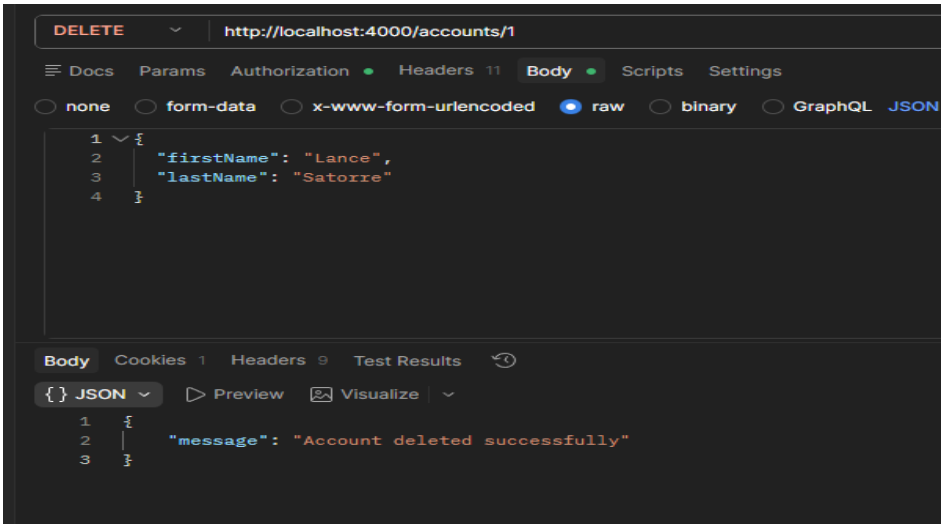
`503f041af7f83e8ca3a3efcb31465c60135779a80ec94be633cd9bd7c6f9a17b149b3a0d652f67ae`



7.6 Other Endpoints to Test

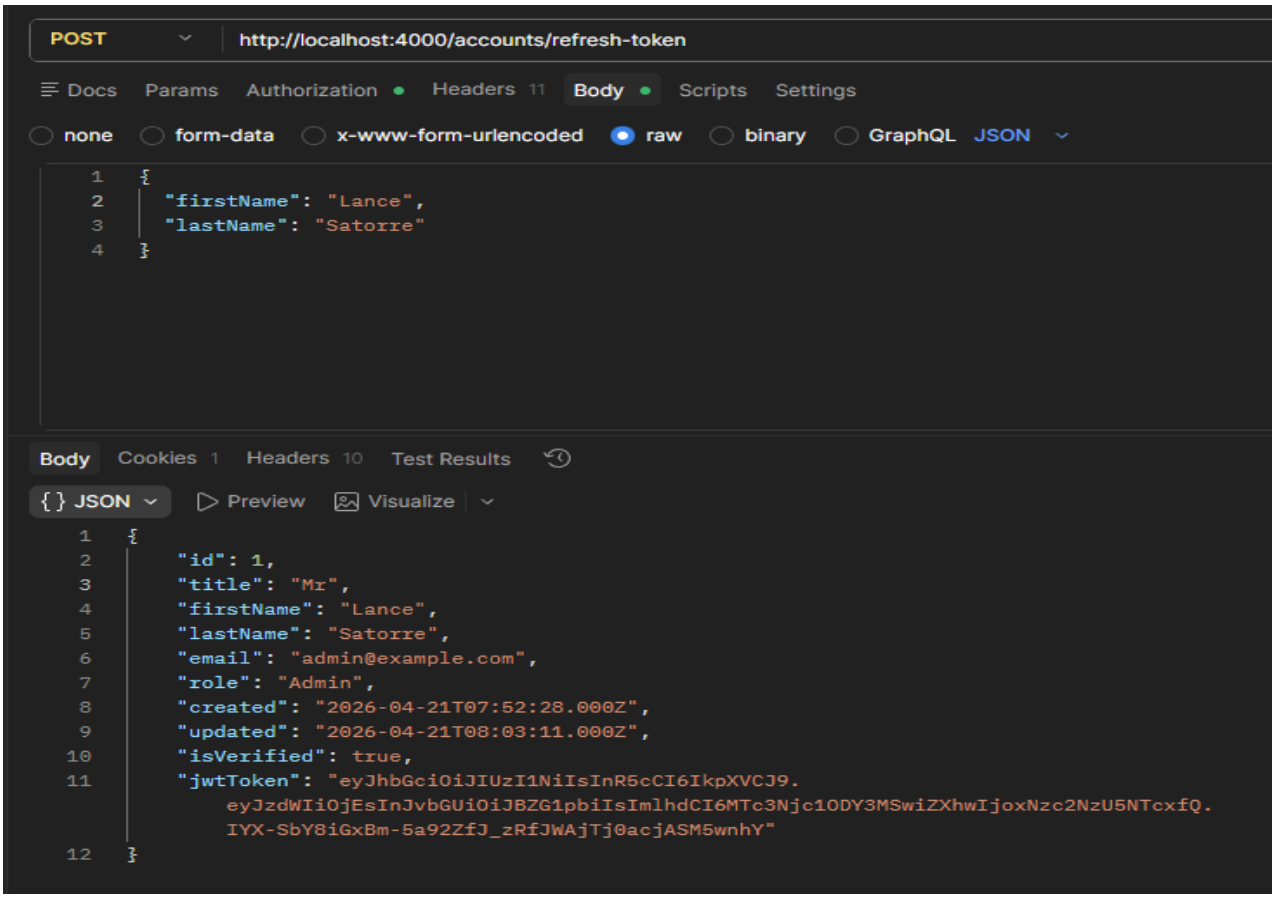
- PUT /accounts/:id — update account (Bearer token required)
- POST /accounts/refresh-token — rotates token, returns new JWT
- POST /accounts/revoke-token — revokes a refresh token
- DELETE /accounts/:id — deletes account (Admin or own account)





7.7 Verify in MySQL

```
mysql -u root -p
USE node_boilerplate;
SELECT id, email, role, isVerified FROM accounts;
SELECT id, accountId, revoked FROM refreshTokens;
```



Body Cookies 1 Headers 10 Test Results 200 OK • 40 ms • 825 B

Name	Value	Domain	Path	Expires	HttpOnly	Secure
refreshToken	e091c34e1cc066...	localhost	/	Tue, 28 Apr 2026...	true	false

API Route Reference

Method	Endpoint	Description
POST	/accounts/register	Register new account
POST	/accounts/verify-email	Verify email token
POST	/accounts/authenticate	Login → get JWT
POST	/accounts/refresh-token	Rotate refresh token
POST	/accounts/forgot-password	Send reset email
POST	/accounts/reset-password	Set new password
POST	/accounts/revoke-token	Revoke refresh token [Auth]
GET	/accounts	List all accounts [Admin]
GET	/accounts/:id	Get account by ID [Auth]
POST	/accounts	Admin create account [Admin]
PUT	/accounts/:id	Update account [Auth]
DELETE	/accounts/:id	Delete account [Auth]